# Assignment No: 6

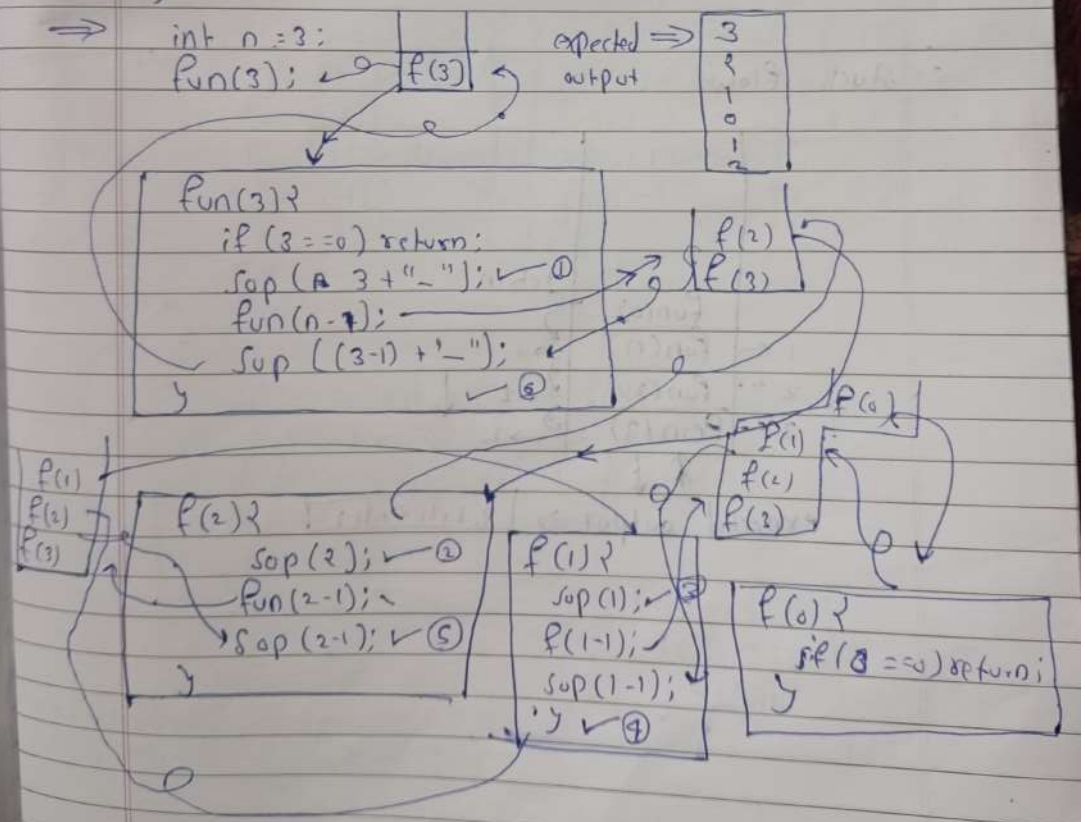
Instructions → ① Don't run the code immediately
② First dry run on paper then predict output

Q.1.

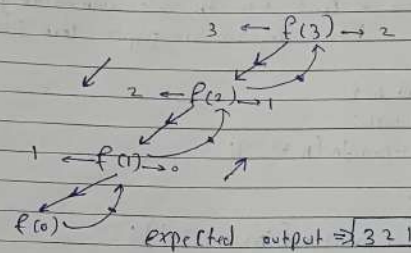
```
void fun (int n){
    if (n==0) return;
    System.out.println (n +"_");

    fun(n-1);
    System.out.println ((n+1) + "_");
}
```

⇒ int n = 3;
fun(3);

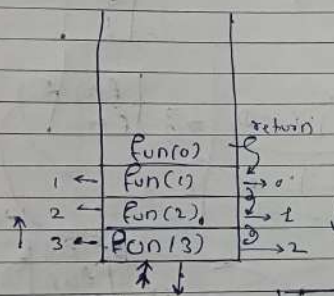


- Recursive tree.




expected output ⇒ 3 2 1 0 1 2

- Stack flow




expected output ⇒ 3, 2, 1, 0, 1, 2

Q3

```
Void fun(int n){
    if(n==0) return;

    Sop("X");
    fun(n-1);
    Sop("Y");
    fun(n-1);
}
```

```
int n = 2
fun(2);
```

```
fun(2){
    if(2==0) return;
    Sop(X) ← ①
    fun(2-1);
    Sop(Y); ← ②
    fun(2-1);
}
```

```
fun(1){
    if(1==0) return
    Sop(X) ← ③
    fun(1-1);
    Sop(Y) ← ④
    fun(1-1);
}
```

```
fun(0){
    if(0==0){
        return;
    }
```

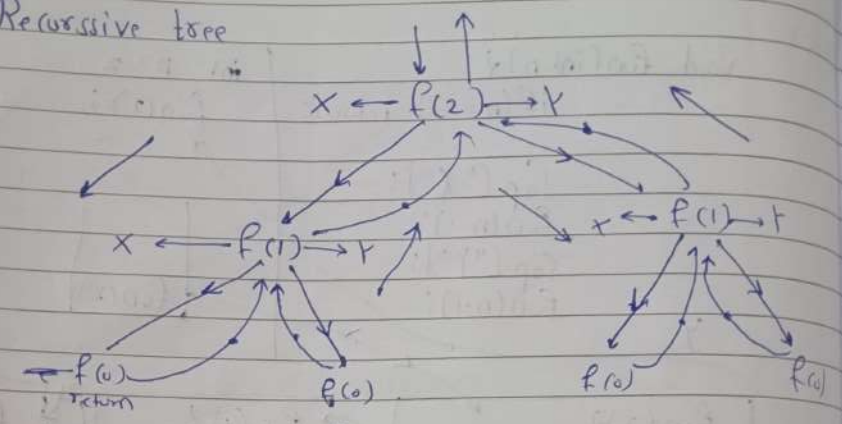
```
fun(1){
    if(1==0) return;
    Sop(X) ← ⑤
    fun(1-1);
    Sop(Y); ← ⑥
    fun(1-1);
}
```

```
fun(0){
    if(0==0){
        return;
    }
```

```
fun(0){
    if(0==0){
        return;
    }
```

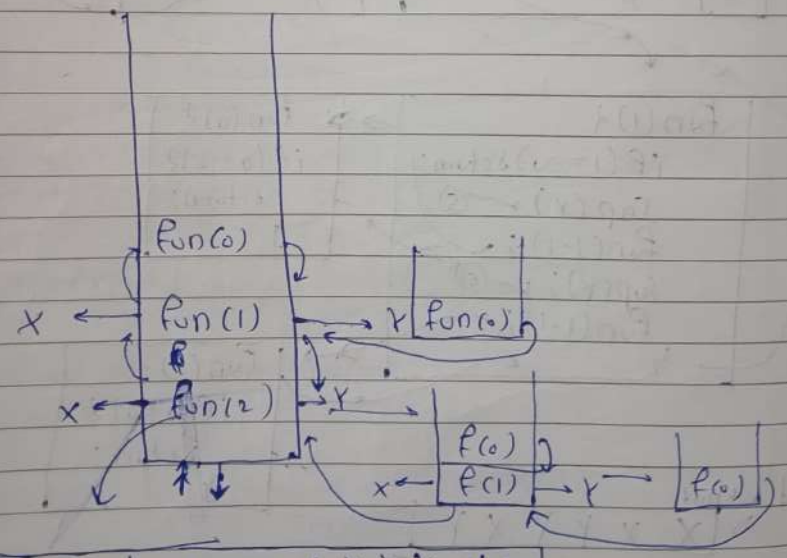
expected output ⇒ X X Y Y X Y

★ Recurssive tree




★ expected output ⇒ X X Y Y X Y

★ Stack flow




★ expected output ⇒ X X Y Y X Y

Q.3.

```
void fun(int n){
if (n<=1){
    sop(n);
    return;
}
fun(n-2);
sop(n);
fun(n-2);
}
```

```
int n = 4;
fun(4);
```

f(4)

```
fun(4){
  if (4<=1)x
  fun(n-2);
  sop(n); ✓ ④
  fun(n-2);
```

```
fun(2){
  if(2<=1) x
  fun(n-2);
  sop(n); ✓ ②
  fun(n-2);
}
```

```
fun(0){
  if (0<1){
    sop(0); ✓ ①
    return; ✓ ③
  }
```

```
fun(2){
  x if(2<=1)
  fun(0);
  sop(2); ✓ ⑤
  fun(0);
}
```

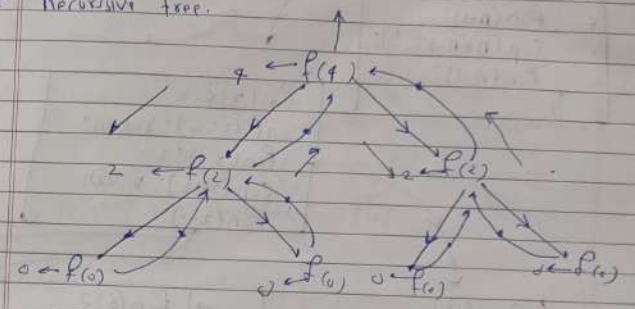
```
fun(0){
  if (0<1){
    sop(0) ✓ ⑥
    return;
  }
```

```
fun(0){
  if (0<1){
    sop(0) ✓ ⑦
    return;
  }
```

expected output

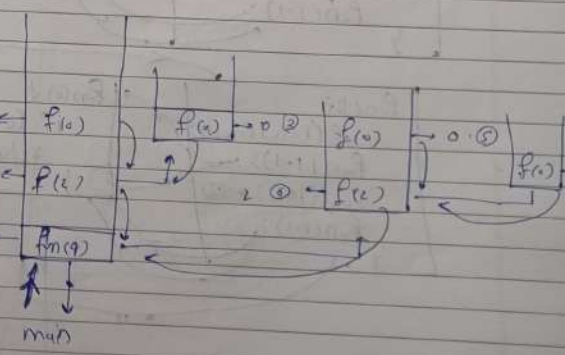
⇒ 0 2 0 4 0 2 0

A. Recursive tree.




expected output ⇒ 0 2 0 4 0 2 0

A. Stack flow.




expected expected output ⇒ 0 2 0 4 0 2 0

```
void fun (int n){
    if (n==0) return;
    fun(n-1);
    sop (n*n + "_");
    fun(n-1);
}
```

```
int n=2
fun(2);
```

```
fun(2){
    if (2==0): return;
    fun(2-1); ✓
    sop(2*2): ✓ ②
    fun(2-1);
```

```
fun (1){
    if (1==0) return;
    fun (1-1); ✓
    sop(1*1); ①
    fun(1-1);
}
```

```
fun(0){
    if (0==0){
        return;
    }
```

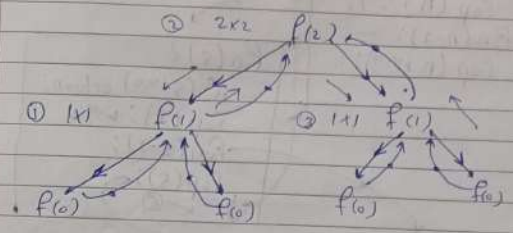
```
fun(1){
    if (1==0) return;
    fun(1-1);
    sop(1*1); ✓ ②
    fun(1-1);
}
```

```
fun(0){
    if (0==0){
        return;
    }
```

- Expected output ⇒

| |
|---|
| 1 |
| 4 |
| 1 |

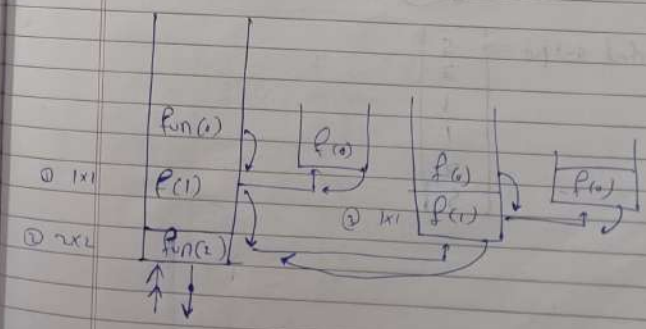
☆ Recurssive tree




- Expected output ⇒

| |
|---|
| 1 |
| 4 |
| 1 |

☆ Stack flow




- Expected output ⇒

| |
|---|
| 1 |
| 4 |
| 1 |

Q.5

```
void fun(int n){
    if (n<=0) return;
    sop (n + "_");
    fun(n-2);
    sop (n + " ");
}
```

```
int n = 5
fun(5);
```

```
fun(5){
    if (5<=0) return;
    sop(5); ✓ ①
    fun(5-2);
    sop(5);
}  ✓ ⑥
```

```
fun(3){
    if (3<=0) return
    sop(3); ✓ ②
    fun(3-2);
    sop(3);
}  ✓ ⑤
```

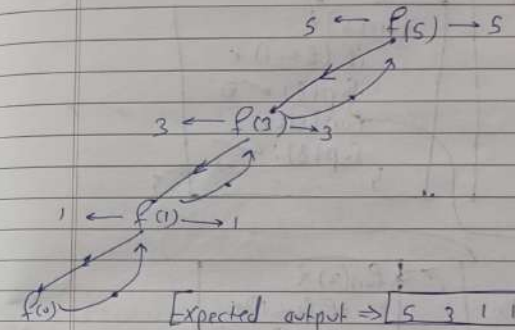
```
fun(1){
    if (1<=0) return
    sop(1); ✓ ③
    fun(1-2);
    sop(1);
}  ✓ ④
```

```
fun(0){
    if (0<=0){
        return;
    }
```

Expected output ⇒

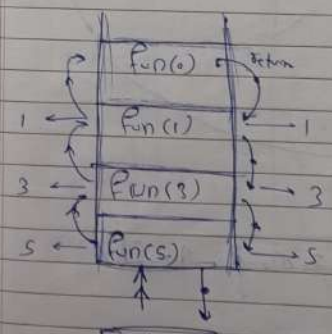
| |
|---|
| 5 |
| 3 |
| 1 |
| 1 |
| 3 |
| 5 |

★ Recursive Tree




Expected output ⇒ [5 3 1 1 3 5]

★ Stack ~~over~~ flow




Expected output ⇒ [5 3 1 1 3 5]

Q. 6

```
void fun(int n){
    if (n<=1){
        sop(n);
        return;
    }
    sop(n);
    fun(n/2);
    sop(n);
}
```

```
int n=8;
fun(n);
```

```
fun(8){
    if (8<=1) x
    sop(8) ✓ ①
    fun(8/2);
    sop(8); ✓ ⑦
}
```

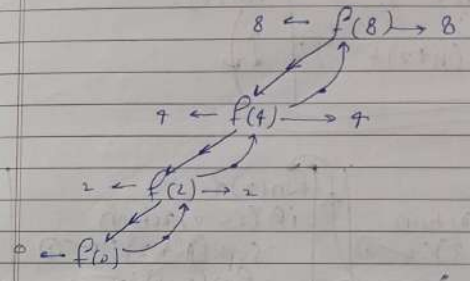
```
fun(4){
    if (4<=1) x
    sop(4) ✓ ②
    fun(4/2); ✓
    sop(4); ✓ ⑥
}
```

```
fun(2){
    if (2≤1) x
    sop(2); ✓ ③
    fun(2/2);
    sop(2); ✓ ⑤
}
```

```
fun(0){
    if (0≤1){
        sop(0); ✓ ④
        return;
    }
}
```

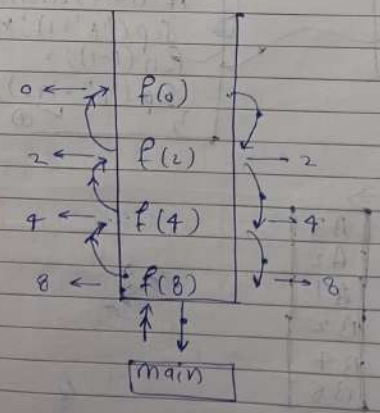
Expected output ⇒ $\boxed{8, 4, 2, 0, 2, 4, 8}$

★ Recurssive Tree ⇒




Expected output ⇒ $\boxed{8, 4, 2, 0, 2, 4, 8}$

★ Stack Flow



Q.7

```
void fun(int n){
    if (n<=0) return;
    Sop ("A"+n+" ");
    fun(n-1);
    Sop("B"+(n*2)+" ");
}
```

```
int n=3
fun(3);
```

```
fun(3){
    if(3<=0) return
    Sop ("A"+3); ✓ ①
    fun(3-1);
    Sop ("B"+3x2);
    ✓ ⑤
}
```

```
fun(2){
    if (2<=0) return
    Sop ("A"+2); ✓ ②
    fun(2-1);
    Sop ("B"+2x2);
    ✓ ⑤
}
```

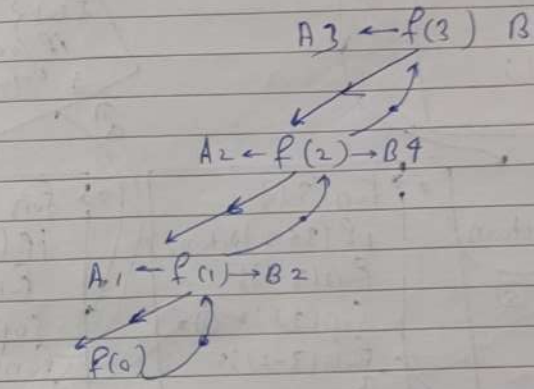
```
fun(1){
    if (1<=0) return
    Sop ("A"+1); ✓ ③
    fun(1-1);
    Sop ("B"+1x2);
    ✓ ④
}
```

```
fun(0){
    if (0<=0){
        return;
    }
}
```

Expected output ⇒

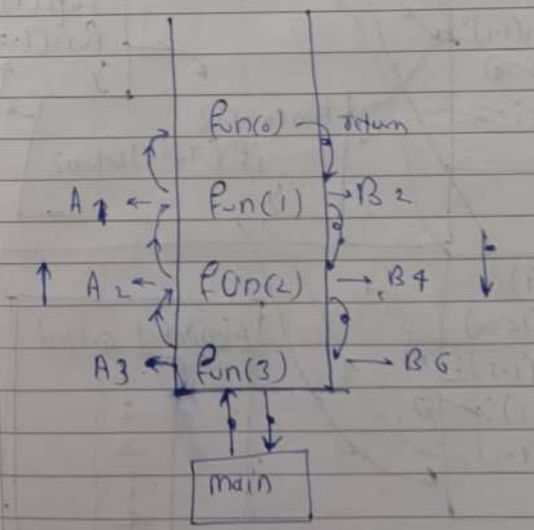
| |
|---|
| A3 |
| A2 |
| A1 |
| B2 |
| B4 |
| B6 |

★ Recursion Tree




- Expected output ⇒ A3, A2, A1, B2, B4, B6

★ Stack Flow




Expected output ⇒ A3, A2, A1, B2, B4, B6